

Examen práctico

Sistema de Biblioteca: microservicios que se descubren, orquestan y persisten

Dato	Detalle
Docente	Nike Rodríguez
Modalidad	Individual · remoto
Entrega	Repositorio de GitHub (privado; agregar al docente como colaborador)
Fecha límite	Domingo 19 de julio de 2026, 11:59 p.m.
Sustentación	Semana del 21 al 23 de julio (7pm)
Calificación	Sobre 20 puntos

1. Objetivo del examen

Este examen evalúa si puedes aplicar, en un dominio nuevo (una Biblioteca), los conceptos de descomposición en microservicios: descubrimiento de servicios, comunicación por nombre con balanceo de carga, y persistencia real con JPA + PostgreSQL.

Además, refuerza tres prácticas de ingeniería que todo backend developer debe manejar: patrones de diseño, pruebas unitarias y análisis de calidad de código. No se trata de copiar un proyecto de referencia cambiando nombres: se trata de demostrar que entendiste el patrón de arquitectura y lo puedes reproducir solo, en un dominio distinto.

2. ¿Qué va a permitir hacer este sistema?

Antes de entrar en el detalle técnico, esto es lo que vas a construir, sin tecnicismos:

Imagina que este sistema es la versión digital de una biblioteca. Va a permitir:

- Llevar un registro de los libros (ejemplares): su título, autor, y si están disponibles o prestados en este momento.
- Llevar un registro de los socios: las personas que tienen permiso para pedir libros prestados, y si ese permiso está activo o no.
- Cuando alguien quiere pedir un libro prestado, el sistema revisa automáticamente DOS cosas antes de decir que sí: que el socio esté habilitado, y que el libro esté disponible.
- **Si ambas cosas están bien:** registra el préstamo, marca el libro como "ya no disponible" y le avisa al socio que su préstamo quedó listo.
- **Si algo no está bien** (el socio no puede pedir prestado, el libro ya se lo llevó otra persona, o el libro no existe): el sistema debe decir claramente por qué no se pudo, sin romperse ni mostrar un error feo.

- Cuando alguien devuelve el libro, el sistema lo marca como devuelto y el libro vuelve a estar disponible para el siguiente socio.

Todo esto no lo hace un solo programa gigante. Lo hacen varios programas pequeños e independientes (a esto se le llama microservicios), cada uno encargado de UNA sola responsabilidad, que se avisan entre ellos por internet — como si fueran distintas oficinas de la misma biblioteca que se comunican por teléfono en vez de estar todas apretadas en el mismo cuarto.

Tu trabajo en este examen: construir esas "oficinas" (los servicios), hacer que sepan encontrarse entre sí, y que la conversación entre ellas tenga sentido incluso cuando algo sale mal.

3. Alcance del examen

3.1 Qué SÍ se evalúa

- Diseño y despliegue de 4 microservicios propios, con el patrón maestro → orquestador → notificador aplicado al dominio de Biblioteca.
- Registro y descubrimiento de servicios con Eureka.
- Comunicación entre servicios exclusivamente POR NOMBRE (RestClient.Builder + @LoadBalanced), sin URLs quemadas.
- Que tu sistema (Eureka + tus 3 servicios de negocio) se integre SIN FRICCIÓN con el API Gateway que usará el docente en tu sustentación (ver sección 4).
- Persistencia con JPA + PostgreSQL: una base de datos independiente por servicio de negocio, con campos de auditoría.
- Reglas de negocio con manejo de errores que NO tumben el sistema.
- Patrones de diseño creacionales (Factory Method y Builder).
- Pruebas unitarias con JUnit 5 + Mockito.
- Análisis de calidad de código con SonarCloud.

3.2 Qué NO se evalúa esta vez

Fuera de alcance

Circuit Breaker / Resilience4j · Construcción del API Gateway (te lo entrega el docente) · Config Server · Docker Compose / contenedores. Si ya conoces alguno de estos temas y quieres incluirlo como mérito adicional, coordínalo con el docente antes de la sustentación.

4. Arquitectura del sistema completo

El sistema completo tiene 5 piezas. Tú construyes 4: el registro (Eureka) y los 3 servicios de negocio. La quinta pieza, el API Gateway, te la entrega el docente ya construida — y viene con seguridad.

#	Servicio	Puerto	Rol	¿Quién lo construye?
1	eureka-server	8761	Registro central: aquí se anuncian y se preguntan por los demás	Tú
2	api-gateway	8080	Única puerta pública, con seguridad	El docente (te lo entrega antes de la sustentación)
3	libros-service	8081	El "Maestro": gestiona Ejemplares y Socios	Tú
4	prestamos-service	8082	El orquestador: valida, notifica y registra el préstamo	Tú
5	notificaciones-service	8083	Recibe un aviso y lo "envía" (simulado)	Tú

Importante sobre el API Gateway

No construyes el Gateway. El docente te lo entregará antes de tu sustentación, ya armado y con seguridad (autenticación). Tu trabajo es que TUS 4 servicios sigan EXACTAMENTE los nombres (spring.application.name), puertos y rutas de este documento, para que el Gateway se conecte a tu sistema sin ningún ajuste de tu parte. Antes de la sustentación se te compartirá cómo autenticarte (usuario/token) para probar tus endpoints a través de él.

El flujo general del sistema:

```

Cliente
  | (a través del Gateway del docente, con seguridad)
  ▼
api-gateway —lb://→ libros-service (:8081)      [Ejemplar, Socio]
               └─lb://→ prestamos-service (:8082)
                   │ └─lb://→ libros-service (valida socio y ejemplar)
                   └─lb://→ notificaciones-service (:8083)

Los 4 servicios que tu construyes se registran en eureka-server (:8761)

```

5. Especificación funcional

5.1 Entidades

Servicio	Entidad	Clave primaria	Campos de negocio	Auditoría
libros-service	Ejemplar	codigoEjemplar (String, ej. "BIB-0001")	titulo, autor, isbn, anioPublicacion, disponible	fechaCreacion, fechaActualizacion
libros-service	Socio	codigoSocio (String, ej. "S001")	nombre, email, telefono, fechaInscripcion, activo	fechaCreacion, fechaActualizacion
prestamos-service	Prestamo	id (autogenerado)	codigoEjemplar, codigoSocio, fechaPrestamo, fechaDevolucionEsperada, fechaDevolucionReal, estado, motivoRechazo, observaciones	fechaCreacion, fechaActualizacion

Servicio	Entidad	Clave primaria	Campos de negocio	Auditoría
notificaciones-service	Notificacion	id (autogenerado)	destino, mensaje, canal, estado, fechaEnvio	fechaCreacion, fechaActualizacion

Importante: Ejemplar y Socio deben tener PK NATURAL (no autoincremental). Prestamo y Notificacion sí pueden usar PK autogenerada, porque son el registro de un evento, no un catálogo maestro. Los campos de auditoría (fechaCreacion, fechaActualizacion) son distintos a los campos de negocio (por ejemplo fechaPrestamo): los de auditoría los llena el framework solo, nunca los llena a mano el programador.

5.1.1 Ejemplo de cómo podría quedar cada entidad (guía de referencia, no una plantilla obligatoria)

Puedes ajustar nombres de paquete o agregar anotaciones de validación, siempre que mantengas los campos obligatorios de la tabla anterior.

Ejemplar (libros-service):

```
@Entity
@Table(name = "ejemplares")
@Data @NoArgsConstructor @AllArgsConstructor
public class Ejemplar {

    @Id
    private String codigoEjemplar;        // PK natural, ej. "BIB-0001"

    private String titulo;
    private String autor;
    private String isbn;
    private Integer anioPublicacion;
    private boolean disponible;

    @CreationTimestamp
    @Column(updatable = false)
    private LocalDateTime fechaCreacion;    // se llena sola al crear

    @UpdateTimestamp
    private LocalDateTime fechaActualizacion; // se actualiza sola al editar
}
```

Socio (libros-service):

```
@Entity
@Table(name = "socios")
@Data @NoArgsConstructor @AllArgsConstructor
public class Socio {

    @Id
    private String codigoSocio;            // PK natural, ej. "S001"

    private String nombre;
    private String email;
    private String telefono;
    private LocalDate fechaInscripcion;
    private boolean activo;

    @CreationTimestamp
    @Column(updatable = false)
    private LocalDateTime fechaCreacion;

    @UpdateTimestamp
    private LocalDateTime fechaActualizacion;
}
```

Prestamo (prestamos-service):

```
@Entity
@Table(name = "prestamos")
@Data @NoArgsConstructor @AllArgsConstructor
public class Prestamo {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String codigoEjemplar;
    private String codigoSocio;
    private LocalDateTime fechaPrestamo;
    private LocalDate fechaDevolucionEsperada;
    private LocalDateTime fechaDevolucionReal; // null hasta que se devuelva
    private String estado; // REGISTRADA / RECHAZADA / DEVUELTO
    private String motivoRechazo; // null si no fue rechazada
    private String observaciones;

    @CreationTimestamp
    @Column(updatable = false)
    private LocalDateTime fechaCreacion;

    @UpdateTimestamp
    private LocalDateTime fechaActualizacion;
}
```

Notificacion (notificaciones-service):

```
@Entity
@Table(name = "notificaciones")
@Data @NoArgsConstructor @AllArgsConstructor
public class Notificacion {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String destino;
    private String mensaje;
    private String canal; // "EMAIL" / "SMS" (simulado)
    private String estado; // "ENVIADO"
    private LocalDateTime fechaEnvio;

    @CreationTimestamp
    @Column(updatable = false)
    private LocalDateTime fechaCreacion;

    @UpdateTimestamp
    private LocalDateTime fechaActualizacion;
}
```

5.2 Endpoints — libros-service (:8081) · recurso /api/v1/libros

Método	Endpoint	Descripción
POST	/api/v1/libros	Crear un ejemplar nuevo
GET	/api/v1/libros	Listar todos los ejemplares
GET	/api/v1/libros/{codigoEjemplar}	Buscar uno → 404 tipado si no existe
PUT	/api/v1/libros/{codigoEjemplar}	Editar título, autor o disponibilidad

Método	Endpoint	Descripción
DELETE	/api/v1/libros/{codigoEjemplar}	Eliminar un ejemplar
PATCH	/api/v1/libros/{codigoEjemplar}/disponibilidad	Cambiar solo disponible (true/false) — lo usa prestamos-service internamente

5.3 Endpoints — Socios (mismo servicio, libros-service) · recurso /api/v1/socios

Método	Endpoint	Descripción
POST	/api/v1/socios	Registrar un socio nuevo
GET	/api/v1/socios	Listar todos los socios
GET	/api/v1/socios/{codigoSocio}	Buscar uno → 404 tipado si no existe
PUT	/api/v1/socios/{codigoSocio}	Editar nombre, email o estado activo
DELETE	/api/v1/socios/{codigoSocio}	Eliminar un socio

5.4 Endpoints — prestamos-service (:8082) · recurso /api/v1/prestamos

Método	Endpoint	Descripción
POST	/api/v1/prestamos	Registrar un préstamo: valida socio y ejemplar, notifica, persiste
GET	/api/v1/prestamos	Listar todos los préstamos
GET	/api/v1/prestamos/{id}	Buscar uno por id
POST	/api/v1/prestamos/{id}/devolucion	Marcar DEVUELTO y volver a poner el ejemplar disponible (OBLIGATORIO)

5.5 Endpoints — notificaciones-service (:8083) · recurso /api/v1/notificaciones

Método	Endpoint	Descripción
POST	/api/v1/notificaciones	Recibir y "enviar" (simulado) un aviso
GET	/api/v1/notificaciones	Listar todas las notificaciones

5.6 Rutas que ya trae configuradas el Gateway del docente

No necesitas construir el Gateway. Estas son las rutas que YA vienen configuradas en el Gateway que usará el docente en tu sustentación. Tu única responsabilidad es que tus endpoints respondan exactamente en estos paths, con estos nombres de servicio registrados en Eureka.

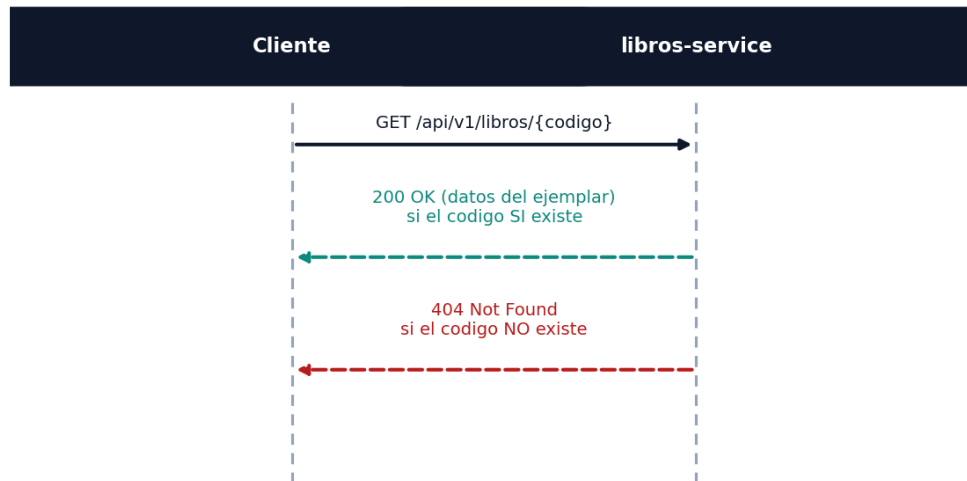
Path (predicate)	Destino esperado (nombre de TU servicio)
/api/v1/libros/**	lb://libros-service
/api/v1/socios/**	lb://libros-service
/api/v1/prestamos/**	lb://prestamos-service
/api/v1/notificaciones/**	lb://notificaciones-service

6. Cómo se comunican los servicios (diagramas de secuencia)

Estos diagramas muestran, paso a paso y en orden, qué mensaje manda cada servicio y en qué momento. Sirven para que entiendas el orden exacto de las llamadas, incluyendo qué pasa cuando algo sale mal.

6.1 libros-service — Consultar un ejemplar (o un socio)

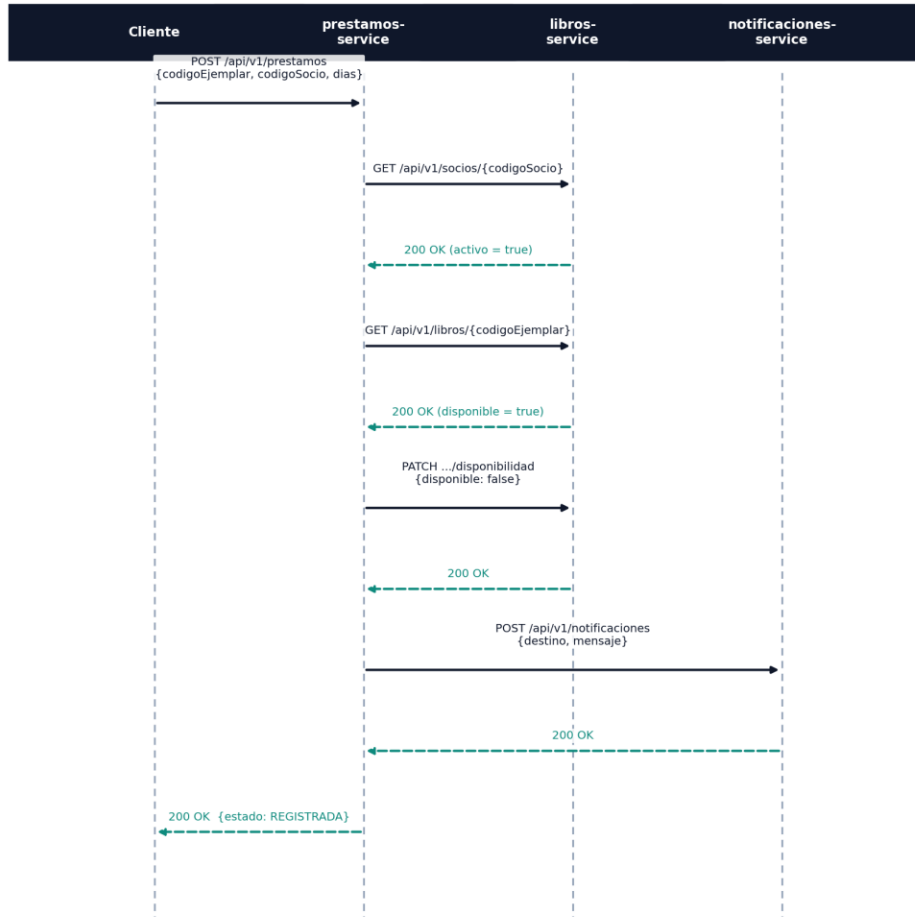
libros-service · Consultar un ejemplar (o un socio)



Cuando alguien pide el detalle de un libro o un socio, libros-service responde con los datos (200) si existe, o avisa con un 404 si el código no existe. El mismo patrón aplica igual para `/api/v1/socios/{codigo}`.

6.2 prestamos-service — Registrar un préstamo (camino exitoso)

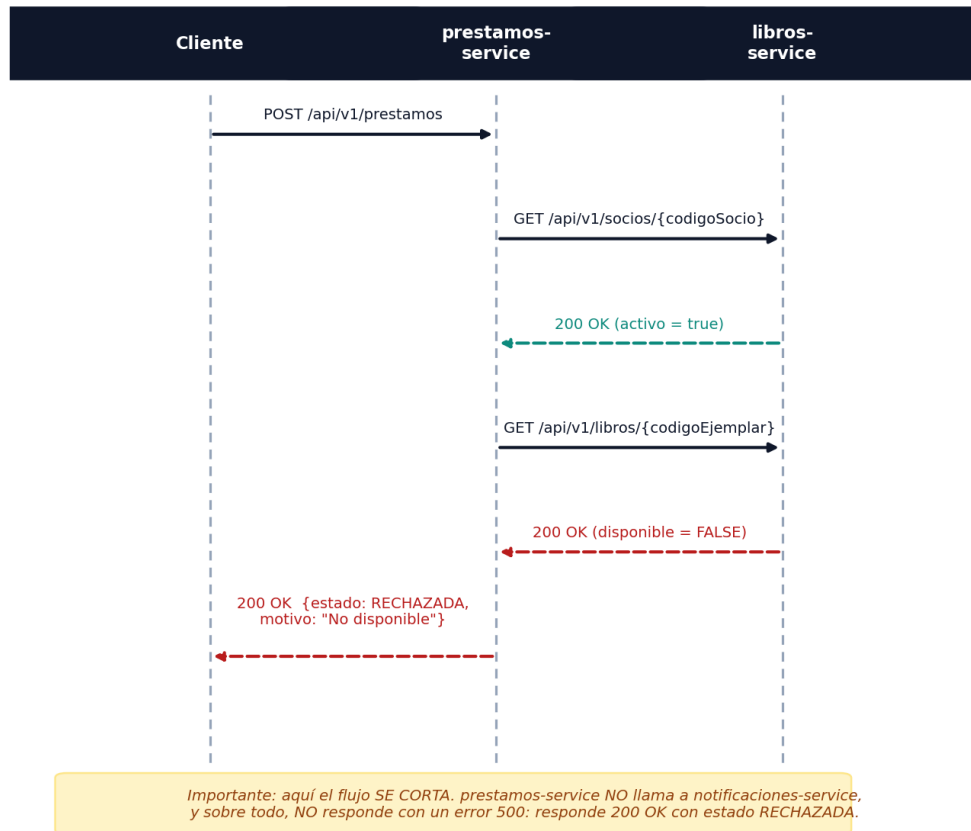
prestamos-service · Registrar un préstamo (camino exitoso)



Este es el camino feliz: el socio existe y está activo, el libro existe y está disponible. prestamos-service coordina tres llamadas (validar socio, validar libro, avisar) antes de responder al cliente.

6.3 prestamos-service — Camino de rechazo (manejo de errores)

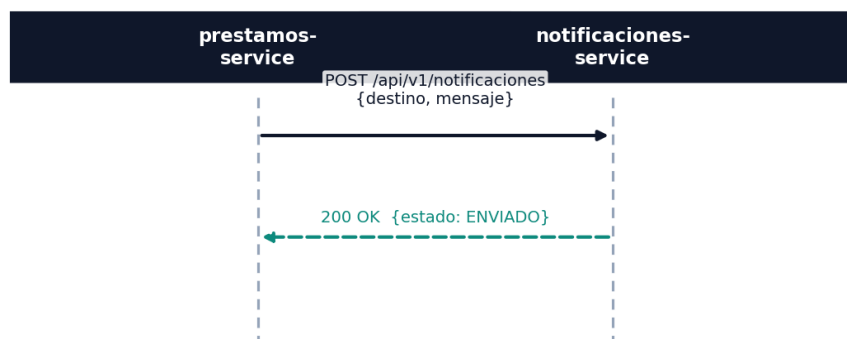
prestamos-service · Camino de rechazo (manejo de errores)



Aquí el libro YA estaba prestado (*disponible=false*). Fíjate que prestamos-service NO llega a llamar a notificaciones-service: corta el flujo apenas detecta el problema, y responde con una explicación clara (estado RECHAZADA) en vez de un error 500. La misma idea aplica para un socio inactivo o un código que no existe.

6.4 notificaciones-service — Recibir y registrar un aviso

notificaciones-service · Recibir y registrar un aviso



El servicio más simple del sistema: recibe un aviso, lo guarda, y confirma. Basta con simular el envío (por ejemplo con un log) y dejar constancia en la base de datos.

7. Sembrado inicial obligatorio

libros-service debe sembrar datos de prueba al arrancar (CommandLineRunner), porque el flujo de la sección 8 depende de ellos:

- **Mínimo 3 ejemplares**, con al menos uno en disponible = false.
- **Mínimo 2 socios**, con al menos uno en activo = false.

8. Flujo de prueba obligatorio

Estos escenarios se ejecutan a través del Gateway del docente durante la sustentación. Son, además, la base de tu colección de Postman y tus pruebas unitarias.

#	Escenario	Resultado esperado
1	GET /api/v1/libros	Lista los ejemplares sembrados
2	GET /api/v1/socios	Lista los socios sembrados
3	POST /api/v1/prestamos con un ejemplar disponible y un socio activo	estado REGISTRADA; el ejemplar queda disponible=false; se genera una notificación
4	Repetir el POST anterior sobre el MISMO ejemplar, con otro socio	estado RECHAZADA, motivo "No disponible"
5	POST /api/v1/prestamos usando el socio inactivo	estado RECHAZADA, motivo "Socio inactivo"
6	POST /api/v1/prestamos con un código de ejemplar inexistente	estado RECHAZADA, motivo "Ejemplar no existe"
7	POST /api/v1/prestamos/{id}/devolucion sobre el préstamo del paso 3	estado DEVUELTO; el ejemplar vuelve a disponible=true
8	Repetir la devolución del paso 7 sobre el mismo préstamo	debe rechazarse: el préstamo ya estaba devuelto
9	GET /api/v1/notificaciones	Debe listar la notificación generada en el paso 3

9. Manejo de errores esperado

Esta tabla resume, para cada situación problemática, quién la debe manejar y qué respuesta se espera. Ningún camino de error debe terminar en un código 500 sin controlar.

Situación	¿Quién la maneja?	Qué debe pasar
GET a un ejemplar con código inexistente	libros-service	404 + excepción propia (ej. EjemplarNoEncontradoException)
GET a un socio con código inexistente	libros-service	404 + excepción propia (ej. SocioNoEncontradoException)
POST /prestamos con socio inactivo	prestamos-service	200 OK, estado=RECHAZADA, motivo="Socio inactivo" (nunca un 500)
POST /prestamos con ejemplar no disponible	prestamos-service	200 OK, estado=RECHAZADA, motivo="No disponible"

Situación	¿Quién la maneja?	Qué debe pasar
POST /prestamos con ejemplar o socio inexistente	prestamos-service	Captura el 404 de libros-service y responde 200 OK, estado=RECHAZADA (no lo propaga como 500)
Devolver un préstamo ya devuelto	prestamos-service	Rechazo controlado (por ejemplo 409 Conflict o un estado de error explícito) — nunca un 500
libros-service no responde (por ejemplo, está apagado)	prestamos-service	Captura el error de conexión con un try/catch y responde RECHAZADA, no revienta con un 500

10. Requisitos técnicos obligatorios

- Los 4 servicios que construyes registrados en Eureka: spring.application.name correcto en cada uno, visibles en estado UP en http://localhost:8761.
- Comunicación entre servicios exclusivamente por NOMBRE: un RestClient.Builder propio, anotado con @LoadBalanced. Cero URLs con localhost o IP quemadas en el código.
- Los nombres de servicio, puertos y paths de tus endpoints deben coincidir EXACTAMENTE con la especificación de la sección 5, para que el Gateway del docente los enrute sin necesitar ningún ajuste de tu parte.
- Persistencia con JPA + PostgreSQL: una base de datos independiente por servicio de negocio (librosdb, prestamosdb, notificacionesdb).
- Manejo de "no encontrado" con excepciones tipadas (@ResponseStatus o @ControllerAdvice).
- El servicio orquestador (prestamos-service) no debe caerse ante un fallo de sus llamadas: si libros-service no responde o el recurso no existe, el préstamo se registra como RECHAZADA, no se propaga un error 500.

11. Patrones de diseño creacionales (obligatorio, mínimo 2)

11.1 Factory Method

Ubicación sugerida: en prestamos-service, una clase MensajeNotificacionFactory que arme el texto del mensaje según el resultado (REGISTRADA, RECHAZADA o DEVUELTO). El service le pide el mensaje a la fábrica en vez de concatenar strings a mano en varios lugares del código.

11.2 Builder

Ubicación sugerida: la construcción del objeto de respuesta del préstamo (PrestamoResponse), que cambia de campos según el camino (registrado, rechazado o devuelto). Al menos UNA implementación del patrón debe estar escrita a mano (sin la anotación @Builder de Lombok), para demostrar que entiendes el patrón y no solo la anotación.

11.3 Documentación obligatoria de cada patrón

- Un comentario en el código: // [Patrón: Factory Method] (o Builder, según corresponda).
- Un párrafo en el README explicando POR QUÉ se usó ahí — no basta con definir el patrón, hay que justificar la decisión de diseño.

12. Pruebas unitarias (obligatorio, mínimo 6)

JUnit 5 + Mockito. No se exige Testcontainers ni pruebas de integración contra la base de datos real: son pruebas UNITARIAS, con mocks del repositorio y de los clientes HTTP. Casos sugeridos (puedes usar otros, siempre que cubran reglas de negocio reales):

- LibroServiceTest: encuentra un ejemplar existente / lanza excepción tipada si no existe.
- PrestamoServiceTest: préstamo REGISTRADO / RECHAZADO por no disponible / RECHAZADO por socio inactivo / RECHAZADO por ejemplar inexistente / devolución exitosa / devolución duplicada rechazada.
- MensajeNotificacionFactoryTest: verifica que arma el mensaje correcto según el estado (prueba pura, sin mocks).

13. Análisis de calidad — SonarCloud (obligatorio)

Aplica sobre los 3 servicios de negocio: libros-service, prestamos-service y notificaciones-service (no se exige sobre eureka-server).

- 1. Crear una cuenta gratuita en sonarcloud.io (login con tu cuenta de GitHub).
- 2. Importar cada repositorio/proyecto a SonarCloud.
- 3. Generar un token y guardarlo como secret SONAR_TOKEN en la configuración de tu repositorio de GitHub.
- 4. Agregar un GitHub Action en .github/workflows/sonarcloud.yml que ejecute el análisis en cada push.
- 5. Evidencia obligatoria en el README: el badge del Quality Gate + el enlace al dashboard del proyecto.

Se exige que el Quality Gate quede en PASSED. Si queda en FAILED, explica en el README qué issues quedaron pendientes y por qué — esto también se evalúa, como honestidad técnica.

14. Entregable

Repositorio de GitHub (puede ser privado; agrega al docente (nike_1109@outlook.es) como colaborador), con esta estructura:

```
apellido-nombre-examen-microservicios/  
├── eureka-server/  
├── libros-service/  
├── prestamos-service/  
├── notificaciones-service/  
├── postman/  
│   └── Biblioteca.postman_collection.json  
├── docs/  
│   ├── eureka-dashboard.png  
│   └── sonarcloud-quality-gate.png  
└── README.md
```

El README.md debe incluir:

- Cómo levantar las 3 bases de datos PostgreSQL (ver Anexo A).
- Orden de arranque de tus 4 servicios.
- Explicación de los patrones de diseño usados (sección 11.3).
- Evidencia de SonarCloud (badge + enlace).
- Instrucciones o capturas para reproducir el flujo de la sección 8.

Anexo A — Levantar las 3 bases de datos

Este examen no incluye Docker Compose. Si prefieres no instalar PostgreSQL local, estos 3 comandos sueltos levantan las bases rápido:

```
docker run --name libros-db -e POSTGRES_DB=librosdb -e POSTGRES_USER=libros \
-e POSTGRES_PASSWORD=libros123 -p 5432:5432 -d postgres:16-alpine

docker run --name prestamos-db -e POSTGRES_DB=prestamosdb -e POSTGRES_USER=prestamos \
-e POSTGRES_PASSWORD=prestamos123 -p 5433:5432 -d postgres:16-alpine

docker run --name notif-db -e POSTGRES_DB=notificacionesdb -e POSTGRES_USER=notif \
-e POSTGRES_PASSWORD=notif123 -p 5434:5432 -d postgres:16-alpine
```

15. Rúbrica de evaluación (20 puntos)

Criterio	Puntos
1. Arquitectura general (tus 4 microservicios: Eureka + 3 de negocio; roles correctos; todos UP en Eureka)	2
2. Comunicación por nombre, sin URLs quemadas (@LoadBalanced)	2
3. Integración sin fricción con el Gateway del docente (nombres, puertos y rutas exactos)	2
4. Persistencia JPA + PostgreSQL correcta (3 bases independientes, con auditoría)	2
5. Regla de negocio del préstamo (los 3 caminos de rechazo)	2
6. Devolución de préstamo (incluye bloqueo de doble devolución)	2
7. Patrones creacionales (Factory Method + Builder, justificados)	2
8. Pruebas unitarias (mínimo 6, casos con sentido de negocio)	2
9. SonarCloud (Quality Gate + evidencia en el README)	2
10. README y estructura del repositorio	1
11. Sustentación oral	1

Total: 20 puntos.

16. Sustentación

Dato	Detalle
Duración	5 minutos por alumno
Fecha	Semana del 21 y 23 de julio (7pm)
Formato	Compartes pantalla, muestras el dashboard de Eureka con tus 4 servicios en UP, ejecutas en vivo 1-2 escenarios de la sección 8 a través del Gateway del docente, y respondes una pregunta puntual sobre tu propio código

Dato	Detalle
Carácter	Individual y obligatoria: un proyecto sin sustentación no se califica, sin importar cuán completo esté

Sobre el Gateway durante la sustentación

El docente conectará tu sistema (Eureka + tus 3 servicios de negocio) a SU API Gateway (con seguridad) y probará en vivo el flujo de la sección 8 a través de él. Por eso es crítico que los nombres, puertos y rutas coincidan exactamente con este documento. Se compartirá antes de la fecha cómo autenticarte contra ese Gateway (usuario/token).

Ejemplo del tipo de pregunta que se hace en el momento: "¿qué pasa ahora mismo si apagas libros-service e intentas registrar un préstamo?" — se espera que expliques tu propio manejo de errores, no que lo repitas de memoria.

17. Honestidad académica

Se permite usar herramientas de inteligencia artificial como apoyo para entender conceptos o resolver dudas puntuales. Sin embargo, la sustentación es individual y debe demostrar comprensión real del propio código: no basta con que el sistema funcione, debes poder explicar por qué funciona así. Un sistema que corre pero cuyo autor no puede sustentarlo se evaluará como incompleto.

18. Resumen de fechas clave

Hito	Fecha
Entrega en GitHub	Domingo 19 de julio de 2026, 11:59 p.m.
Sustentación	Semana del 21 y 23 de julio (7pm)